

# Architecture & Data Model

## Principles

- **One Postgres, two jobs.** Neon holds relational data *and* the pgvector RAG store. pgvector is solid to ~50M vectors — far beyond agency scale, so no separate vector DB.
- **client\_id is the tenancy key.** It threads through every table, query, and retrieval filter. A brain can never read another brain's data — and it's the seam that becomes per-customer separation when productized.
- **Cost is first-class.** Every paid action writes a row attributing spend to a stage, a production, a user, and a client. Nothing calls a paid API without passing the gate.
- **The loop is closed.** Published assets and their performance live in the model, so brains

learn from outcomes, not just from documents.

## Schema (Neon / Postgres + pgvector)

```
create extension if not exists
vector;

create table users (
  id          uuid primary key default
gen_random_uuid(),
  email       text unique not null,
  name        text,
  role        text not null default
'producer', -- 'admin' | 'producer'
  created_at  timestampz default
now()
);

-- A "brain" = a client. The dropdown
lists these.
create table clients (
  id          uuid primary key
default gen_random_uuid(),
  name        text not null,
  -- 'GAS PSI', 'Andrew Eaton',
```

```

'Lernalot'
  slug                text unique not
null,
  status              text not null
default 'active',
  brand               jsonb not null
default '{}', -- logo_url, colors,
fonts, lower_third style
  sonic_identity      jsonb not null
default '{}', -- music style
descriptor / ElevenLabs finetune id
  voice_id            text,
-- ElevenLabs voice (BY0)
  heygen_avatar_id    text,
  higgsfield_soul_id  text,
  created_at          timestampz
default now()
);

-- Authoritative, hand-locked truth.
Always injected. Partly seeded from
the GAS Sheet. Versioned.
create table client_profiles (
  id                  uuid primary
key default gen_random_uuid(),
  client_id           uuid not null
references clients(id) on delete
cascade,
  version             int not null

```

```

default 1,
  positioning      text,
  audience         jsonb,
  banned_words     text[] default
'{}',
  tone_rules       text,
  proof_points     jsonb,
  outcome_definitions text,
-- what a "win" looks like for THIS
client
  exemplars        jsonb,
-- best content + winning patterns
(fed by learning loop)
  is_live          boolean default
false,            -- human-approved before
use
  created_at       timestampz
default now()
);

-- Knowledge sources: client website,
or a GAS Google Sheet pointing to a
Drive folder.
create table knowledge_sources (
  id               uuid primary key
default gen_random_uuid(),
  client_id        uuid not null
references clients(id) on delete
cascade,

```

```

    type          text not null,
-- 'website' | 'gsheet'
    uri           text not null,
-- site root URL, or Google Sheet ID
    status        text not null
default 'pending', -- pending |
indexed | failed
    last_synced_at timestampz,
    created_at    timestampz default
now()
);

-- RAG store. NEVER queried without a
client_id filter.
create table knowledge_chunks (
    id            uuid primary key default
gen_random_uuid(),
    client_id     uuid not null references
clients(id) on delete cascade,
    source_id     uuid not null references
knowledge_sources(id) on delete
cascade,
    content       text not null,
    embedding     vector(1536),
    metadata      jsonb default '{}'
-- {title, url, tags[]}
);
create index on knowledge_chunks
using hnsu (embedding

```

```

vector_cosine_ops);
create index on knowledge_chunks
(client_id);

-- One production run.
create table productions (
    id            uuid primary key
default gen_random_uuid(),
    client_id     uuid not null
references clients(id),
    created_by    uuid not null
references users(id),
    title         text,
    brief         jsonb not null,
-- topic, segment, options, tier,
resolution, voice mode
    plan         jsonb,
-- script + scenes + captions +
popups + audio_map
    duration_target int not null
default 45,
    status        text not null
default 'draft',
    -- draft | estimating |
awaiting_approval | rendering_draft |
draft_ready
    --      | awaiting_final_approval |
rendering_final | qa_review |
complete | failed | cancelled

```

```

    estimate_cents int,
    actual_cents   int default 0,
    draft_video_url text,
    final_video_url text,
    created_at     timestampz default
now()
);

-- Per-stage cost attribution (the
dashboard's source of truth).
create table production_steps (
    id            uuid primary key
default gen_random_uuid(),
    production_id uuid not null
references productions(id) on delete
cascade,
    stage         text not null, --
'tts','music','ambient','aroll','brol
l','assemble_draft','assemble_final'
    provider      text not null,
    units         numeric,
    cost_cents    int,
    output_ref    text,
    status        text not null default
'pending',
    started_at    timestampz,
    finished_at   timestampz
);

```

```

-- Editable price list. Prices live
HERE, never in code.
create table rate_card (
  id                uuid primary
key default gen_random_uuid(),
  provider          text not null,
  model             text,
  unit              text not null,
  --
  'char','second','clip','render_minute
','minute','token'
  resolution        text,
  price_cents_per_unit numeric not
null,
  effective_from    timestampz
default now(),
  active            boolean
default true
);

-- Spend guardrails enforced BEFORE
paid calls.
create table budgets (
  id                uuid primary key
default gen_random_uuid(),
  scope             text not null,
-- 'user' | 'client' | 'team'
  scope_id          uuid,
  period            text not null default

```

```

'monthly',
  limit_cents int not null,
  spent_cents int not null default 0,
  hard_gate   boolean not null
default true,      -- block (true)
vs alert-only (false)
  created_at  timestampz default
now()
);

-- LEARNING LOOP: where a finished
video went live.
create table published_assets (
  id          uuid primary key
default gen_random_uuid(),
  production_id uuid not null
references productions(id),
  client_id    uuid not null
references clients(id),
  platform     text not null,
-- 'facebook','tiktok','x','linkedin'
  url          text,
  segment      text,
-- audience segment targeted
  posted_at    timestampz,
  created_at   timestampz default
now()
);

```

```
-- LEARNING LOOP: performance pulled
back via campaign-analyst skills.
create table performance_metrics (
  id          uuid primary key
default gen_random_uuid(),
  asset_id    uuid not null
references published_assets(id) on
delete cascade,
  platform    text not null,
  metric      text not null,
--
'retention','ctr','engagement','conve
rsions'
  value       numeric,
  captured_at timestampz default
now()
);
```

Consent records ( `consents` ,  
`consent_texts` ) live in `compliance.md` —  
every photo/voice  
upload writes one, with a  
date+time+timezone stamp, for  
POPIA/GDPR.

## Isolation rules (non-negotiable)

1. Every `knowledge_chunks` query carries `where client_id = $active`. No exceptions.
2. Retrieval, profile load, voice/avatar/Soul/sonic-identity selection all derive from the dropdown's active `client_id`. The brain governs the *whole* pipeline.
3. Blob keys namespaced `clients/{client_id}/...` so files are partitioned too.

## Productization seams (present now, light up later)

- **Tenancy:** `client_id` everywhere → per-customer separation is a filter, not a rewrite.
- **Branding + sound:** `clients.brand` + `clients.sonic_identity` → white-label look *and* sound.
- **Billing:** `production_steps.cost_cents` + `rate_card` → add a markup column → client invoices.

- **Value story for sale:**

`performance_metrics` proves the brains improve outcomes over time — the productized pitch isn't "we make videos," it's "your content compounds."

- **Currency:** USD base with per-user/per-tenant display currency (ZAR now; GBP/AUD/etc. as you scale) — see `cost-controls.md`. International markets are a display + FX-pair add, not a rebuild.